



UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

LA CONCURRENCIA EN LOS SISTEMAS OPERATIVOS

Fundamentos, mecanismos de sincronización y problemas clásicos

Sistemas operativos

EQUIPO 1

Grupo: 003 | Hora: Martes V4 – V6 | Aula: 1303

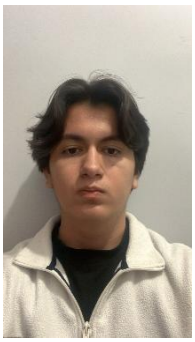
https://youtu.be/pdhba3-7j8I

Profesora: Dra. Marín Martínez Norma Edith.

Fecha de entrega: 18 de agosto de 2026

Integrantes del equipo

Matrícula	Nombre	Programa	Foto	Porcentaje de participación
2144053	Lozano Garza Camila del Carmen	IAS		100%
2160859	Vera Moreno Oscar Elias	ITS		100%
2226699	Sánchez Santiago Osmar Abdi	ITS		100%
2178081	Cisneros Moreno Valeria Nohemí	ITS		100%

2144813	Torres Guerrero David Alonso	ITS		100%
2065934	Gonzalez Belloc Yael Ricardo	ITS		100%
2161066	Ramirez Fabian Maria de los Angeles	IAS		100%
2169669	Sánchez Juárez Emiliano	ITS		100%
2097304	Jaime Sebastian Esparza Leyva	ITS		100%

2158392	Martinez Tamez Julio César	IAS		100%
2135792	Muñoz Hernández Salvador	IAS		100%
1320176	Maycott López Gilberto	IAS		100%

Índice

1. Introducción 4
2. Fundamentos de la concurrencia 5
   2.1 Concepto de proceso y de hilo 5
   2.2 Motivación: ¿por qué la concurrencia? 6
   2.3 Concurrencia frente a paralelismo 6
   2.4 Estados de un proceso y planificación 7
   2.5 Sistemas multinúcleo y la ley de Amdahl 8
3. El problema de la sección crítica 9
   3.1 Condiciones de carrera 9
   3.2 Regiones críticas y exclusión mutua 10
   3.3 Requisitos de una solución correcta 10
4. Soluciones basadas en software 11
   4.1 Variables de candado y alternancia estricta 11
   4.2 La solución de Peterson 12
5. Soluciones basadas en hardware 12
   5.1 Deshabilitación de interrupciones 12
   5.2 Instrucciones atómicas: TSL y XCHG 13
   5.3 Espera ocupada y candados de giro 14
6. Mecanismos de sincronización de alto nivel 14
   6.1 Semáforos 14
   6.2 Mutexes 15
   6.3 Variables de condición 15
   6.4 Monitores 16
   6.5 Transmisión de mensajes 16
   6.6 Barreras 17
   6.7 Comparación de los mecanismos 17
7. Problemas clásicos de concurrencia 17
   7.1 El problema del productor-consumidor 18
   7.2 El problema de los lectores y escritores 19
   7.3 El problema de los filósofos comelones 19
8. Interbloqueos 21
   8.1 Las cuatro condiciones de Coffman 21
   8.2 Modelado con grafos de asignación de recursos 21
   8.3 Estrategias de manejo 22
   8.4 Inanición y bloqueo activo 23
9. Concurrencia en los sistemas operativos modernos 23
   9.1 Concurrencia en Linux 23
   9.2 Concurrencia en Windows 24
   9.3 Tendencias y desafíos 24
10. Conclusión 25

Referencias bibliográficas 26

Apéndice de imágenes y tablas

El presente apéndice reúne el material gráfico y tabular empleado a lo largo de la investigación. Cada elemento se identifica con la nomenclatura de su leyenda, de modo que pueda localizarse con facilidad dentro del cuerpo del documento.

Índice de imágenes

concurrencia-imagen 1 Concurrencia frente a paralelismo en uno y varios núcleos.

concurrencia-imagen 2 Diagrama de estados de un proceso.

concurrencia-imagen 3 Condición de carrera al incrementar un contador compartido.

concurrencia-imagen 4 Esquema del problema del productor-consumidor con búfer acotado.

concurrencia-imagen 5 El problema de los filósofos comelones.

concurrencia-imagen 6 Grafo de asignación de recursos con un ciclo de interbloqueo.

Índice de tablas

concurrencia - Tabla 1 Diferencias entre concurrencia y paralelismo.

concurrencia - Tabla 2 Requisitos de una solución al problema de la sección crítica.

concurrencia - Tabla 3 Comparación de los mecanismos de sincronización.

concurrencia - Tabla 4 Principales primitivas de hilos de POSIX (Pthreads).

concurrencia - Tabla 5 Problemas clásicos de comunicación entre procesos.

concurrencia - Tabla 6 Las cuatro condiciones de Coffman para el interbloqueo.

1. Introducción

Desde sus orígenes, el propósito esencial de un **sistema operativo** ha sido administrar de manera eficiente los recursos de la computadora y ofrecer a los programas una máquina extendida más simple que el hardware desnudo. Con el paso de las décadas, la evolución del hardware —en particular la aparición de los procesadores de varios núcleos— transformó a la concurrencia, antes un tema periférico, en uno de los ejes centrales del diseño de los sistemas operativos modernos (Tanenbaum, 2009).

La **concurrencia** es la propiedad de un sistema por la cual múltiples actividades —procesos o hilos— progresan durante intervalos de tiempo que se solapan. Esta capacidad hace posible que un usuario navegue por la Web mientras se reproduce música y se descarga correo electrónico, o que un servidor atienda miles de peticiones simultáneas sin bloquearse. Sin embargo, la concurrencia introduce una clase de dificultades que no existen en los programas estrictamente secuenciales: cuando varias actividades comparten datos, el resultado de la ejecución puede depender del orden exacto en que se entrelazan sus instrucciones, dando lugar a errores sutiles, no deterministas y difíciles de reproducir (Arpaci-Dusseau y Arpaci-Dusseau, 2014).

El estudio de la concurrencia es, por tanto, doblemente importante. Por un lado, permite aprovechar el paralelismo del hardware para mejorar el rendimiento; por el otro, obliga a comprender y dominar los mecanismos de **sincronización** que garantizan la corrección de los programas. Como advierten Silberschatz, Galvin y Gagne (2018), la ejecución concurrente de procesos cooperativos exige que el sistema provea herramientas para coordinar el acceso a los recursos compartidos; de lo contrario, la integridad de los datos queda comprometida.

La presente investigación documental tiene como objetivo ofrecer una visión integral y ordenada de la concurrencia en los sistemas operativos. Para ello se recurre a tres obras de referencia en la materia: *Sistemas operativos modernos* de Andrew S. Tanenbaum, *Operating System Concepts* de Abraham Silberschatz y colaboradores, y *Operating Systems: Three Easy Pieces* de Remzi y Andrea Arpaci-Dusseau. A partir de ellas se construye un recorrido que parte de los fundamentos conceptuales, examina el problema de la sección crítica, describe las soluciones basadas en software y en hardware, analiza los mecanismos de sincronización de alto nivel, revisa los problemas clásicos de comunicación entre procesos, aborda el fenómeno del interbloqueo y, finalmente, comenta cómo se materializa la concurrencia en los sistemas operativos contemporáneos.

La estructura del trabajo sigue un orden pedagógico: cada tema se apoya en el anterior, de modo que el lector avance desde las nociones más elementales

hasta los mecanismos más elaborados. A lo largo del texto se incorporan diagramas y tablas —catalogados en el apéndice— que sintetizan las ideas centrales y facilitan su comprensión.

2. Fundamentos de la concurrencia

2.1 Concepto de proceso y de hilo

El concepto más importante que provee un sistema operativo es el de **proceso**, que puede definirse informalmente como un programa en ejecución. Un proceso no es simplemente el código almacenado en disco —una entidad pasiva—, sino una entidad activa que incluye el valor del contador de programa, el contenido de los registros del procesador, su espacio de direcciones (con las secciones de texto, datos, montículo y pila) y el conjunto de recursos que utiliza, como los archivos abiertos (Silberschatz et al., 2018).

Para sostener la ilusión de que varios programas se ejecutan a la vez sobre una única unidad central de procesamiento, el sistema operativo virtualiza la CPU mediante la técnica de **tiempo compartido**: ejecuta un proceso durante un breve intervalo, lo detiene, guarda su estado y cede el control a otro. Este mecanismo, conocido como **cambio de contexto**, requiere almacenar y restaurar toda la información necesaria para reanudar cada proceso como si nunca se hubiera interrumpido (Arpaci-Dusseau y Arpaci-Dusseau, 2014).

Un **hilo** (o hebra) es la unidad básica de utilización de la CPU dentro de un proceso. Mientras que el modelo tradicional asocia a cada proceso un único hilo de control, los sistemas operativos modernos permiten que un proceso contenga varios hilos que se ejecutan de forma cuasi-paralela dentro del mismo espacio de direcciones. Los hilos de un mismo proceso comparten el código, los datos globales y los recursos del proceso, pero cada uno posee su propio contador de programa, su conjunto de registros y su pila (Tanenbaum, 2009). Precisamente por compartir memoria, los hilos constituyen el escenario natural donde surgen los problemas de concurrencia.

La creación y destrucción de hilos es considerablemente más económica que la de procesos, y el cambio de contexto entre hilos de un mismo proceso es más rápido, pues no exige cambiar el mapa de memoria. Por ello, los hilos se han convertido en el mecanismo predilecto para expresar concurrencia dentro de una aplicación, ya sea un servidor Web que atiende peticiones o un procesador de textos que revisa la ortografía en segundo plano mientras el usuario escribe.

2.2 Motivación: ¿por qué la concurrencia?

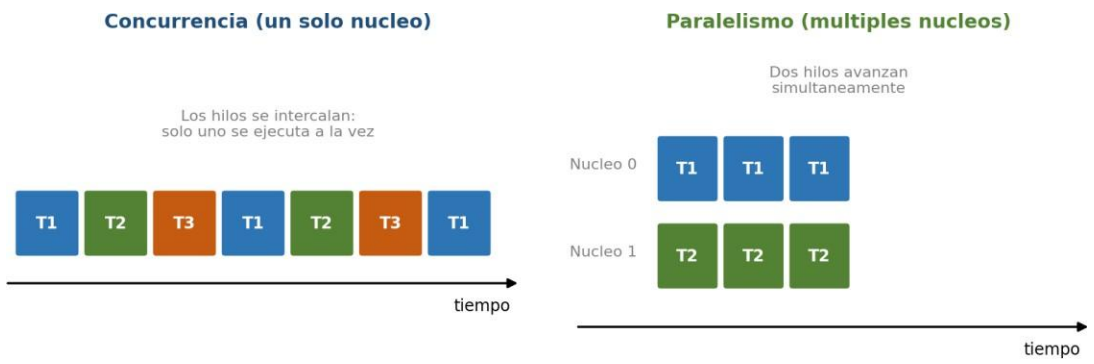
Existen razones de peso para introducir concurrencia en el diseño de aplicaciones y sistemas. Silberschatz et al. (2018) identifican cuatro beneficios principales de la programación multihilo, que pueden extenderse a la concurrencia en general:

1. **Capacidad de respuesta.** Una aplicación interactiva puede seguir atendiendo al usuario aunque una de sus partes esté bloqueada o realizando una operación prolongada, lo que resulta esencial en el diseño de interfaces.
2. **Compartición de recursos.** Los hilos comparten por omisión la memoria y los recursos del proceso al que pertenecen, lo que permite que varias actividades operen sobre los mismos datos sin recurrir a mecanismos costosos de comunicación.
3. **Economía.** Crear y conmutar hilos consume menos tiempo y memoria que crear y conmutar procesos, pues los hilos comparten los recursos del proceso.
4. **Escalabilidad.** En arquitecturas con varios núcleos, los hilos pueden ejecutarse en paralelo sobre procesadores distintos, de modo que el aumento de núcleos se traduce en un incremento real del rendimiento.

A estas razones se suma una motivación histórica: la concurrencia nació dentro del propio sistema operativo. Como observa Tanenbaum (2009), un sistema de multiprogramación mantiene varios procesos en memoria y alterna entre ellos para aprovechar la CPU mientras alguno espera la finalización de una operación de entrada/salida. De este modo, cuando un proceso se bloquea a la espera del disco, otro puede utilizar el procesador, elevando la utilización global del sistema.

2.3 Concurrencia frente a paralelismo

Aunque a menudo se usan como sinónimos, la concurrencia y el paralelismo son conceptos distintos. La **concurrencia** se refiere a que varias tareas progresan durante intervalos solapados; el **paralelismo**, en cambio, implica que varias tareas se ejecutan literalmente al mismo tiempo. En un sistema con un único núcleo solo puede haber concurrencia: el planificador entrelaza las tareas con tanta rapidez que se genera la ilusión de simultaneidad, pero en cada instante se ejecuta una sola. En un sistema con múltiples núcleos es posible el paralelismo verdadero, pues distintos hilos ocupan núcleos diferentes de forma simultánea (Silberschatz et al., 2018). Es posible, por tanto, tener concurrencia sin paralelismo, pero no paralelismo sin concurrencia.



concurrencia-imagen 1. *Concurrencia frente a paralelismo en uno y varios núcleos.*

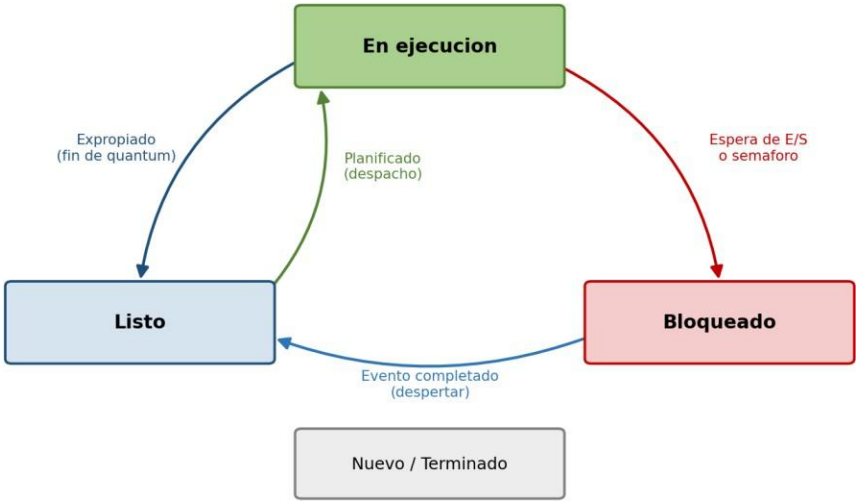
La distinción tiene consecuencias prácticas. En un entorno concurrente pero no paralelo, los errores de sincronización pueden pasar inadvertidos durante mucho tiempo, porque el entrelazado desafortunado de instrucciones ocurre solo bajo ciertas condiciones de planificación. En un entorno paralelo, esos mismos errores se manifiestan con mayor frecuencia, ya que dos hilos pueden acceder a la misma dirección de memoria en el mismo ciclo. La tabla siguiente resume las diferencias esenciales.

Aspecto	Concurrencia	Paralelismo
Definición	Varias tareas progresan en intervalos solapados	Varias tareas se ejecutan a la vez
Hardware necesario	Un solo núcleo es suficiente	Requiere varios núcleos o procesadores
Simultaneidad real	No; se entrelazan las tareas	Sí; se ejecutan de forma simultánea
Objetivo principal	Estructurar y responder a múltiples eventos	Acelerar el cómputo dividiendo el trabajo
Relación	Puede existir sin paralelismo	Implica siempre concurrencia

concurrencia - Tabla 1. *Diferencias entre concurrencia y paralelismo.*

2.4 Estados de un proceso y planificación

Para que la concurrencia sea posible, el sistema operativo modela cada proceso mediante un conjunto de **estados**. En una vista simplificada, un proceso puede encontrarse en ejecución (usando la CPU), listo (preparado para ejecutarse pero a la espera de que el planificador le asigne el procesador) o bloqueado (incapaz de continuar hasta que ocurra un evento externo, como la finalización de una operación de entrada/salida). Las transiciones entre estos estados son gobernadas por el planificador y por los eventos del sistema (Tanenbaum, 2009).



concurrencia-imagen 2. *Diagrama de estados de un proceso.*

Toda la información que el sistema necesita para gestionar un proceso se almacena en una estructura llamada **bloque de control de proceso** (PCB, por sus siglas en inglés), que contiene el estado, el contador de programa, los registros, la información de planificación y de gestión de memoria, y la lista de recursos asignados. El paso de un proceso a otro implica guardar el PCB del proceso saliente y restaurar el del entrante. Es importante observar que la planificación introduce por sí misma un elemento de no determinismo: como el programador no controla cuándo ocurrirá un cambio de contexto, no puede suponer ningún orden particular en el entrelazado de instrucciones de procesos concurrentes. Esta impredecibilidad, benigna cuando los procesos son independientes, se convierte en la raíz de los problemas de concurrencia cuando comparten datos.

2.5 Sistemas multinúcleo y la ley de Amdahl

La proliferación de los chips multinúcleo respondió a un límite físico: resultó cada vez más difícil incrementar la velocidad de un único procesador sin consumir demasiada energía ni generar calor excesivo. En lugar de acelerar un solo núcleo, los arquitectos comenzaron a integrar varios en un mismo circuito. Como consecuencia, el reto se trasladó del hardware al software: para aprovechar esos núcleos, las aplicaciones deben dividirse en tareas que puedan ejecutarse en paralelo (Tanenbaum, 2009).

El grado en que el paralelismo mejora el rendimiento está acotado por la **ley de Amdahl**, que relaciona la aceleración obtenida con la fracción del programa que debe ejecutarse de manera secuencial. Si *S* es la porción estrictamente serial de una aplicación y *N* el número de núcleos, la aceleración máxima es `1 / ( S + (1 - S)/N )`. El corolario es contundente: incluso una fracción secuencial pequeña

impone un techo a la ganancia. Si el 50 % del programa es serial, ningún número de núcleos permitirá superar una aceleración de dos veces (Silberschatz et al., 2018). Por ello, identificar y minimizar las secciones que requieren exclusión mutua —que fuerzan la ejecución serial— es tan importante como distribuir el trabajo entre los núcleos.

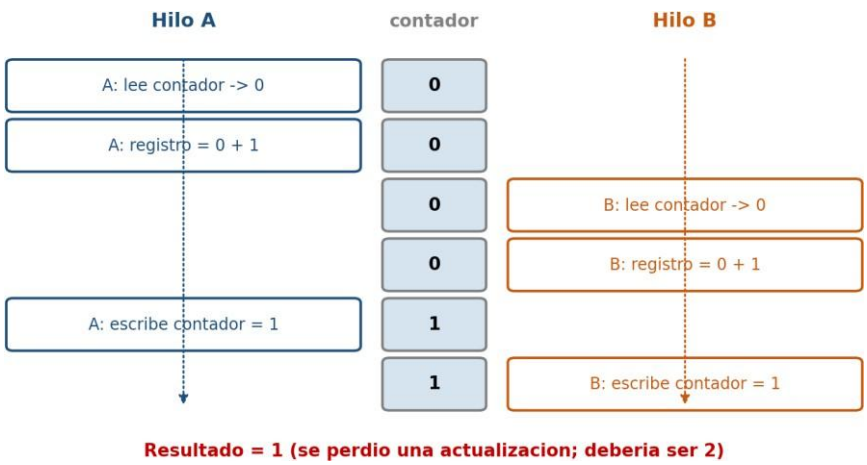
Diseñar programas concurrentes correctos y eficientes plantea, según Silberschatz et al. (2018), cinco desafíos: identificar las tareas que pueden ejecutarse en paralelo, equilibrar la carga entre ellas, dividir adecuadamente los datos, gestionar las dependencias entre datos y, por último, probar y depurar programas cuyo comportamiento no es determinista. Este último punto es especialmente arduo, pues un error puede manifestarse solo en uno de los múltiples entrelazados posibles.

3. El problema de la sección crítica

3.1 Condiciones de carrera

Cuando dos o más procesos leen y escriben datos compartidos, y el resultado final depende del orden preciso en que se ejecutan sus instrucciones, se produce una **condición de carrera**. Tanenbaum (2009) ilustra este fenómeno con el ejemplo del directorio de un gestor de impresión (spooler): dos procesos que intentan encolar un archivo casi al mismo tiempo pueden leer el mismo índice de ranura libre, escribir cada uno el nombre de su archivo en esa ranura y, en consecuencia, uno sobrescribir al otro. El resultado es que un archivo nunca se imprime, y el fallo depende de un entrelazado tan específico que rara vez se reproduce en las pruebas.

Un ejemplo más elemental, presentado por Arpaci-Dusseau y Arpaci-Dusseau (2014), consiste en dos hilos que incrementan un contador compartido. La instrucción de alto nivel `contador = contador + 1` se traduce, en lenguaje máquina, en tres pasos: cargar el valor desde memoria a un registro, incrementar el registro y almacenar el resultado de vuelta en memoria. Como estos tres pasos no se ejecutan de forma atómica, un cambio de contexto en el momento inoportuno puede provocar que ambos hilos lean el mismo valor inicial, lo incrementen por separado y escriban el mismo resultado, de modo que una de las dos actualizaciones se pierde.



concurrencia-imagen 3. *Condición de carrera al incrementar un contador compartido.*

El diagrama anterior muestra un entrelazado concreto en el que dos hilos incrementan un contador que vale cero. Ambos leen el valor 0, ambos calculan 1 en su registro y ambos escriben 1, cuando el resultado correcto debería ser 2. Lo insidioso de este comportamiento es que la mayoría de las ejecuciones producen el resultado esperado; solo un entrelazado desafortunado, difícil de provocar deliberadamente, revela el error. Esta es la razón por la que depurar programas concurrentes resulta tan complejo.

3.2 Regiones críticas y exclusión mutua

La porción de código en la que un proceso accede a recursos compartidos se denomina **región crítica** o sección crítica. La clave para evitar las condiciones de carrera consiste en garantizar la **exclusión mutua**: asegurar que, mientras un proceso ejecuta su región crítica, ningún otro proceso pueda entrar en la suya. Si se logra que dos procesos nunca estén simultáneamente en sus regiones críticas sobre los mismos datos, las carreras desaparecen (Tanenbaum, 2009).

La estructura general de un programa concurrente correcto distingue cuatro secciones: la sección de entrada, donde el proceso solicita permiso para acceder; la región crítica propiamente dicha; la sección de salida, donde libera el acceso; y la sección restante, con el resto del código. El problema de la sección crítica consiste en diseñar los protocolos de entrada y de salida de manera que se cumplan ciertas propiedades fundamentales.

3.3 Requisitos de una solución correcta

No basta con lograr la exclusión mutua; una solución satisfactoria debe cumplir simultáneamente varias condiciones. Combinando las formulaciones de

Tanenbaum (2009) y Silberschatz et al. (2018), pueden enunciarse los siguientes requisitos:

Requisito	Descripción
Exclusión mutua	Dos procesos no pueden estar a la vez dentro de sus regiones críticas sobre el mismo recurso.
Progreso	Solo los procesos que compiten por entrar participan en la decisión, y esta no puede posponerse indefinidamente.
Espera acotada	Ningún proceso debe esperar por siempre para entrar en su región crítica (ausencia de inanición).
Independencia de la velocidad	No pueden hacerse suposiciones sobre la velocidad relativa ni el número de procesadores.
No bloqueo externo	Un proceso que se ejecuta fuera de su región crítica no debe impedir el avance de los demás.

concurrencia - Tabla 2. *Requisitos de una solución al problema de la sección crítica.*

Estos requisitos actúan como criterio de corrección: cualquier mecanismo de sincronización —de software, de hardware o de alto nivel— debe evaluarse según los satisfaga o no. En las secciones siguientes se examinan las distintas familias de soluciones a la luz de estos criterios.

4. Soluciones basadas en software

4.1 Variables de candado y alternancia estricta

La primera idea que suele proponerse es utilizar una única variable compartida de **candado**, inicialmente en cero. Un proceso que desea entrar en su región crítica comprueba el candado: si vale cero, lo pone a uno y entra; si vale uno, espera. Por desgracia, esta solución adolece del mismo defecto que pretende resolver, pues la comprobación y la modificación del candado no son atómicas: un proceso puede leer un cero, ser interrumpido antes de escribir el uno, y permitir que otro proceso también encuentre el cero, de modo que ambos entren en la región crítica (Tanenbaum, 2009).

Una segunda propuesta es la **alternancia estricta**, que emplea una variable de turno para decidir a quién le corresponde entrar. Aunque garantiza la exclusión mutua, viola el requisito de progreso: obliga a los procesos a turnarse de forma rígida, de manera que un proceso que se encuentra en su sección restante puede impedir que otro entre en su región crítica. Además, la técnica se basa en la **espera ocupada**, esto es, en comprobar continuamente una variable dentro de un bucle, lo que desperdicia ciclos de CPU.

4.2 La solución de Peterson

En 1981, G. L. Peterson descubrió una solución de software sencilla y correcta que combina la idea del turno con un arreglo de banderas de interés. Antes de entrar en la región crítica, cada proceso indica su interés activando su bandera y cede el turno al otro; luego espera mientras el otro esté interesado y sea su turno. Este ingenioso mecanismo satisface la exclusión mutua, el progreso y la espera acotada para dos procesos (Tanenbaum, 2009). Su esquema esencial es el siguiente:

```
#define N 2
int turno;
int interesado[N];        // inicialmente en 0 (falso)

void entrar_region(int proceso) {
    int otro = 1 - proceso;
    interesado[proceso] = TRUE;   // muestra interes
    turno = proceso;              // cede el turno
    while (turno == proceso &&
           interesado[otro] == TRUE) ; // espera
}

void salir_region(int proceso) {
    interesado[proceso] = FALSE;  // libera la region
}
```

La solución de Peterson es valiosa desde el punto de vista conceptual, pues demuestra que la exclusión mutua puede lograrse íntegramente por software, sin apoyo especial del hardware. No obstante, presenta dos limitaciones prácticas. La primera es que se basa en espera ocupada, que consume CPU inútilmente y puede provocar el **problema de la inversión de prioridades**, en el que un proceso de alta prioridad gira eternamente esperando a otro de baja prioridad que nunca llega a ejecutarse para liberar el recurso. La segunda es que las arquitecturas modernas reordenan las instrucciones y emplean memorias caché por núcleo, lo que puede invalidar las suposiciones de la solución si no se emplean barreras de memoria adecuadas (Arpaci-Dusseau y Arpaci-Dusseau, 2014). Por estas razones, en la práctica se prefieren las soluciones que combinan apoyo de hardware con intervención del sistema operativo.

5. Soluciones basadas en hardware

5.1 Deshabilitación de interrupciones

En un sistema con un único procesador, la forma más simple de lograr la exclusión mutua es deshabilitar las interrupciones al entrar en la región crítica y volver a habilitarlas al salir. Puesto que el cambio de contexto solo ocurre como consecuencia de una interrupción, un proceso con las interrupciones

deshabilitadas puede examinar y actualizar la memoria compartida sin temor a ser interrumpido (Tanenbaum, 2009).

Sin embargo, esta técnica tiene inconvenientes graves. Otorgar a los procesos de usuario la facultad de deshabilitar las interrupciones es peligroso: un proceso que olvide reactivarlas podría paralizar todo el sistema. Además, en una máquina con varios procesadores, deshabilitar las interrupciones de un núcleo no impide que los demás núcleos accedan a la memoria compartida. Por ello, la deshabilitación de interrupciones es una herramienta útil dentro del propio núcleo del sistema operativo, para proteger la actualización de sus estructuras internas durante unas pocas instrucciones, pero no constituye un mecanismo general apropiado para los programas de usuario.

5.2 Instrucciones atómicas: TSL y XCHG

Las computadoras diseñadas pensando en el multiprocesamiento ofrecen instrucciones especiales que realizan de forma **atómica** una lectura seguida de una escritura sobre una posición de memoria. La instrucción `TSL` (Test and Set Lock) lee el contenido de una palabra de memoria, lo guarda en un registro y almacena un valor distinto de cero en esa palabra, todo ello sin que ningún otro procesador pueda intervenir, pues la CPU bloquea el bus de memoria durante la operación. Una instrucción alternativa, `XCHG`, intercambia atómicamente el contenido de un registro y una palabra de memoria; es la que emplean los procesadores de la familia x86 de Intel para la sincronización de bajo nivel (Tanenbaum, 2009).

Con estas instrucciones puede construirse un candado correcto. El protocolo de entrada gira leyendo y fijando el candado atómicamente hasta que lo encuentra libre; el de salida simplemente escribe un cero:

```
entrar_region:
    TSL  REGISTRO, CANDADO    ; copia candado y lo fija a 1
    CMP  REGISTRO, #0         ; el candado era 0?
    JNE  entrar_region        ; si no, repite (espera ocupada)
    RET                       ; entra a la region critica

salir_region:
    MOVE CANDADO, #0          ; libera el candado
    RET
```

Sobre estas primitivas atómicas se edifican operaciones de nivel superior muy utilizadas en la práctica, como `compare-and-swap` (comparar e intercambiar) y `fetch-and-add` (obtener e incrementar), que permiten implementar contadores y estructuras de datos sin bloqueos. Estas instrucciones son la base sobre la que el sistema operativo construye cerrojos, semáforos y demás mecanismos de sincronización de alto nivel (Arpaci-Dusseau y Arpaci-Dusseau, 2014).

5.3 Espera ocupada y candados de giro

Tanto la solución de Peterson como las basadas en TSL o XCHG comparten un defecto: recurren a la **espera ocupada**. Un candado que hace girar al proceso comprobando repetidamente una variable hasta que puede adquirirlo se denomina **candado de giro** (spinlock). En un multiprocesador, cuando la región crítica es muy breve y la probabilidad de contención baja, girar durante unos pocos ciclos puede ser más eficiente que dormir y despertar al proceso, pues evita el costo de un cambio de contexto (Arpaci-Dusseau y Arpaci-Dusseau, 2014).

No obstante, la espera ocupada desperdicia CPU cuando la espera se prolonga y resulta francamente contraproducente en un sistema monoprocesador, donde el proceso que gira impide que se ejecute el proceso que posee el candado. La solución consiste en sustituir el giro por primitivas que *bloqueen* al proceso: en lugar de consumir ciclos, el proceso que no puede entrar se suspende y cede la CPU, y será despertado cuando el recurso quede libre. Esta idea da lugar a los mecanismos de sincronización de alto nivel que se estudian a continuación.

6. Mecanismos de sincronización de alto nivel

Los mecanismos de alto nivel evitan la espera ocupada suspendiendo a los procesos que no pueden avanzar y despertándolos cuando la condición que esperaban se cumple. Silberschatz et al. (2018) señalan que la clave está en un par de operaciones elementales, `dormir` y `despertar`: la primera bloquea al proceso que la invoca; la segunda reactiva a un proceso previamente bloqueado. A partir de esta idea se construyen semáforos, mutexes, monitores y otros mecanismos.

6.1 Semáforos

En 1965, Edsger W. Dijkstra propuso el **semáforo**, una variable entera que, además de un valor, dispone de dos operaciones atómicas: `down` (también llamada P o wait) y `up` (V o signal). La operación `down` comprueba si el valor es mayor que cero; si lo es, lo decrementa y continúa; si es cero, bloquea al proceso. La operación `up` incrementa el valor y, si había procesos bloqueados, despierta a uno de ellos. La atomicidad de ambas operaciones es indispensable para evitar nuevas condiciones de carrera (Tanenbaum, 2009).

Los semáforos cumplen dos funciones complementarias. Un **semáforo binario**, inicializado en uno, garantiza la exclusión mutua: un proceso ejecuta `down` antes de entrar en la región crítica y `up` al salir. Un semáforo contador, en cambio, sirve para la **sincronización**, esto es, para imponer un orden entre eventos, como asegurar que el consumidor no extraiga de un búfer vacío ni el productor inserte

en uno lleno. Su gran ventaja frente a la espera ocupada es que el proceso bloqueado no consume CPU mientras aguarda.

El principal riesgo de los semáforos es que su uso es propenso a errores. Como advierte Tanenbaum (2009), basta invertir el orden de dos operaciones `down` para provocar un interbloqueo del que el sistema no puede recuperarse. Programar con semáforos se asemeja a programar en lenguaje ensamblador: es poderoso, pero un descuido mínimo puede acarrear fallos de reproducción muy difícil.

6.2 Mutexes

Cuando no se necesita la capacidad de conteo del semáforo, se emplea una versión simplificada llamada **mutex** (del inglés *mutual exclusion*). Un mutex es una variable con dos estados —abierto o cerrado— que resulta ideal para administrar la exclusión mutua sobre un recurso o un fragmento de código. Se manipula con dos operaciones: `mutex_lock`, que adquiere el mutex o bloquea al proceso si ya está cerrado, y `mutex_unlock`, que lo libera y despierta a un proceso en espera. Por su sencillez, los mutexes se implementan de forma muy eficiente, especialmente en las bibliotecas de hilos de espacio de usuario (Tanenbaum, 2009).

La biblioteca de hilos de POSIX, conocida como **Pthreads**, ofrece un conjunto normalizado de primitivas de sincronización que ilustran estos conceptos en la práctica. La tabla siguiente resume las llamadas más importantes relacionadas con mutexes y con variables de condición.

Primitiva de Pthreads	Descripción
pthread_mutex_init	Crea e inicializa un mutex.
pthread_mutex_lock	Adquiere el mutex o bloquea al hilo si está cerrado.
pthread_mutex_trylock	Intenta adquirir el mutex; falla en lugar de bloquear.
pthread_mutex_unlock	Libera el mutex y despierta a un hilo en espera.
pthread_cond_wait	Bloquea al hilo hasta recibir una señal en una condición.
pthread_cond_signal	Despierta a un hilo bloqueado en una variable de condición.
pthread_cond_broadcast	Despierta a todos los hilos bloqueados en la condición.

concurrencia - Tabla 4. *Principales primitivas de hilos de POSIX (Pthreads).*

6.3 Variables de condición

Los mutexes bastan para conceder o denegar el acceso a una región crítica, pero no permiten que un hilo espere hasta que se cumpla cierta condición lógica
—por ejemplo, que un búfer deje de estar vacío—. Para ello, Pthreads ofrece las **variables de condición**, que casi siempre se emplean junto con un mutex. Un hilo que descubre que no puede avanzar invoca `pthread_cond_wait`, operación que

libera atómicamente el mutex y bloquea al hilo; más tarde, otro hilo que modifica el estado ejecuta `pthread_cond_signal` para despertarlo (Silberschatz et al., 2018).

Una diferencia crucial respecto a los semáforos es que las variables de condición **no tienen memoria**: si se señaliza una condición cuando ningún hilo está esperando, la señal se pierde. Por ello, la comprobación de la condición debe realizarse siempre dentro de un bucle `while`, de modo que, al despertar, el hilo vuelva a verificar que la condición se cumple efectivamente antes de continuar. Esta disciplina protege además frente a los llamados despertares espurios.

6.4 Monitores

Dada la facilidad con que se cometen errores usando semáforos, Brinch Hansen y Hoare propusieron en la década de 1970 una primitiva de sincronización de nivel superior: el **monitor**. Un monitor es una construcción del lenguaje de programación que agrupa un conjunto de procedimientos, variables y estructuras de datos en un módulo. Su propiedad fundamental es que **solo un proceso puede estar activo dentro del monitor en cada instante**; el compilador se encarga de imponer esta exclusión mutua, de modo que el programador no tiene que gestionarla explícitamente (Tanenbaum, 2009).

Al convertir cada región crítica en un procedimiento del monitor, se garantiza automáticamente que dos procesos nunca ejecuten sus regiones críticas a la vez. Para permitir que los procesos se bloqueen cuando no pueden continuar, los monitores incorporan variables de condición con operaciones `wait` y `signal`. La ventaja del monitor es que traslada la responsabilidad de la exclusión mutua del programador al compilador, reduciendo drásticamente la probabilidad de error. Su desventaja es que requiere el apoyo del lenguaje: C y muchos otros lenguajes no incorporan monitores, aunque Java sí ofrece un mecanismo equivalente mediante los métodos declarados `synchronized`, que garantizan que un solo hilo ejecute a la vez cualquier método sincronizado de un objeto.

6.5 Transmisión de mensajes

Los semáforos y monitores presuponen memoria compartida, por lo que no sirven en sistemas distribuidos, donde cada procesador posee su propia memoria privada. Para esos entornos —y también dentro de una sola máquina— se emplea la **transmisión de mensajes**, que utiliza dos primitivas: `send` (enviar) y `receive` (recibir). Un proceso envía un mensaje a un destino y otro lo recibe, bloqueándose si aún no hay mensaje disponible (Tanenbaum, 2009).

La transmisión de mensajes plantea retos propios, sobre todo cuando los procesos se comunican a través de una red: los mensajes pueden perderse, por lo que suele emplearse un esquema de acuse de recibo y de números de

secuencia para descartar duplicados. También debe resolverse cómo se nombran los procesos y cómo se autentican mutuamente. Pese a estas complicaciones, el paradigma es muy popular en la programación paralela; un ejemplo destacado es la interfaz MPI (*Message-Passing Interface*), ampliamente utilizada en el cómputo científico. El sistema Mach y el mecanismo Binder de Android son ejemplos de sistemas operativos que adoptan la transmisión de mensajes como forma primaria de comunicación entre procesos (Silberschatz et al., 2018).

6.6 Barreras

Un último mecanismo, orientado a grupos de procesos más que a parejas, es la **barrera**. Algunas aplicaciones —como las simulaciones físicas que avanzan por fases— exigen que ningún proceso pase a la siguiente fase hasta que todos hayan terminado la actual. Para lograrlo, se coloca una barrera al final de cada fase: cuando un proceso llega a ella, se bloquea hasta que todos los demás también hayan llegado; en ese momento, todos se liberan simultáneamente (Tanenbaum, 2009). Las barreras son habituales en el cómputo paralelo de alto rendimiento, donde el trabajo se reparte entre muchos procesos que deben sincronizarse periódicamente.

6.7 Comparación de los mecanismos

Cada mecanismo de sincronización ofrece un equilibrio distinto entre expresividad, eficiencia y facilidad de uso. La tabla siguiente los compara según sus características más relevantes, con el fin de orientar la elección del mecanismo apropiado para cada situación.

Mecanismo	Uso principal	Ventaja	Inconveniente
Candado de giro	Regiones críticas muy breves	Rápido; evita cambio de contexto	Espera ocupada; malo en monoprocesador
Semáforo	Exclusión mutua y sincronización	Versátil; permite conteo	Propenso a errores e interbloqueos
Mutex	Exclusión mutua simple	Eficiente y sencillo	No permite conteo ni orden
Variable de condición	Esperar a que se cumpla una condición	Se combina con el mutex; no hay espera ocupada	Requiere bucle de comprobación
Monitor	Encapsular datos compartidos	El compilador impone la exclusión mutua	Necesita soporte del lenguaje
Mensajes	Sistemas distribuidos	No requiere memoria compartida	Sobrecarga de copia y de protocolo
Barrera	Sincronizar fases de un grupo	Coordina muchos procesos a la vez	Solo aplica a cómputo por fases

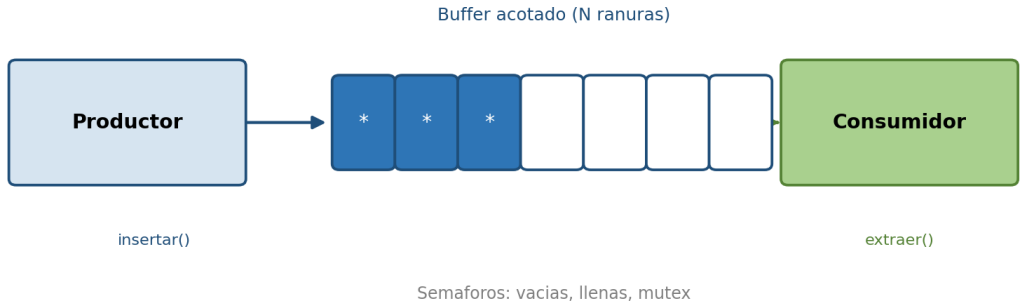
concurrencia - Tabla 3. *Comparación de los mecanismos de sincronización.*

7. Problemas clásicos de concurrencia

La literatura sobre sistemas operativos ha consagrado un conjunto de problemas clásicos que sirven para poner a prueba cualquier mecanismo de sincronización. Su valor es doble: por un lado, modelan situaciones reales de coordinación; por otro, funcionan como banco de pruebas para evaluar la expresividad y la corrección de las primitivas propuestas.

7.1 El problema del productor-consumidor

También conocido como problema del **búfer acotado**, plantea la coordinación entre dos procesos que comparten un búfer de tamaño fijo. El **productor** genera elementos y los deposita en el búfer; el **consumidor** los extrae. El productor debe bloquearse cuando el búfer está lleno, y el consumidor cuando está vacío; además, ambos deben acceder al búfer en exclusión mutua para no corromper sus variables (Tanenbaum, 2009).



concurrencia-imagen 4. *Esquema del problema del productor-consumidor con búfer acotado.*

La solución canónica emplea tres semáforos: `vacias`, que cuenta las ranuras libres e inicializa en *N*; `llenas`, que cuenta las ranuras ocupadas e inicializa en cero; y `mutex`, un semáforo binario que protege el acceso al búfer. El productor ejecuta `down(&vacias)` para reservar una ranura, `down(&mutex)` para entrar en la región crítica, inserta el elemento, y libera con `up(&mutex)` y `up(&llenas)`. El consumidor procede de forma simétrica:

```
void productor(void) {
    while (TRUE) {
        elemento = producir();
        down(&vacias);        // reserva una ranura libre
        down(&mutex);         // entra a la region critica
        insertar(elemento);
        up(&mutex);           // sale de la region critica
        up(&llenas);          // indica una ranura llena
    }
}
```

```
void consumidor(void) {
    while (TRUE) {
        down(&llenas);        // espera un elemento
        down(&mutex);
        elemento = extraer();
        up(&mutex);
        up(&vacias);          // libera una ranura
        consumir(elemento);
    }
}
```

Este ejemplo evidencia la importancia del orden de las operaciones. Si en el productor se invirtieran las dos operaciones `down`, ejecutando primero `down(&mutex)` y luego `down(&vacias)`, un productor que encontrara el búfer lleno se bloquearía *con el mutex en su poder*; el consumidor no podría entonces entrar en la región crítica para vaciar el búfer, y ambos quedarían bloqueados para siempre. Este es un ejemplo clásico de interbloqueo, tema que se abordará más adelante.

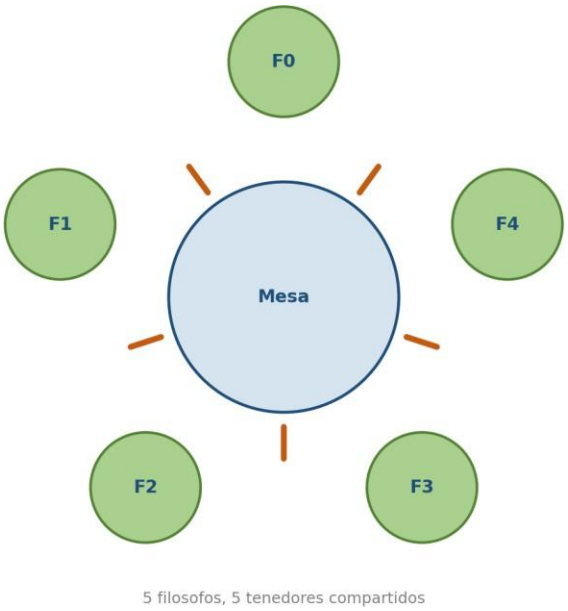
7.2 El problema de los lectores y escritores

El problema de los **lectores y escritores** modela el acceso a una base de datos compartida. Varios procesos lectores pueden consultar los datos simultáneamente sin riesgo, pues no los modifican; en cambio, un escritor requiere acceso exclusivo, ya que una escritura concurrente con cualquier otra operación corrompería la información. El reto consiste en maximizar la concurrencia entre lectores sin comprometer la exclusividad de los escritores (Silberschatz et al., 2018).

Una solución habitual mantiene un contador de lectores activos, protegido por un mutex, y un semáforo que representa el acceso exclusivo a la base de datos. El primer lector que entra adquiere el semáforo del recurso, y el último en salir lo libera; los escritores, por su parte, adquieren directamente ese semáforo. Esta formulación, sin embargo, puede provocar la **inanición** de los escritores: si siempre hay al menos un lector activo, el escritor podría no obtener nunca el acceso. Existen variantes que dan prioridad a los escritores o que garantizan un trato equitativo, según los requisitos de la aplicación.

7.3 El problema de los filósofos comelones

Propuesto por Dijkstra en 1965, el problema de los **filósofos comelones** es quizá el más célebre de todos. Cinco filósofos se sientan alrededor de una mesa circular; entre cada par de platos hay un tenedor, y comer espagueti requiere dos tenedores. La vida de un filósofo alterna entre pensar y comer: cuando tiene hambre, intenta tomar el tenedor de su izquierda y el de su derecha; si lo logra, come y luego deja ambos tenedores (Tanenbaum, 2009).



concurrencia-imagen 5. *El problema de los filósofos comelones.*

La solución ingenua —tomar primero el tenedor izquierdo y luego el derecho—es incorrecta: si los cinco filósofos toman a la vez su tenedor izquierdo, ninguno podrá tomar el derecho y todos quedarán bloqueados, en un interbloqueo perfecto. Reintentar tras liberar el tenedor tampoco resuelve el problema, pues puede dar lugar a **inanición**, una situación en la que los procesos se ejecutan indefinidamente sin progresar. Una solución correcta, que evita tanto el interbloqueo como la inanición y permite el máximo paralelismo, utiliza un arreglo de estados —pensando, hambriento o comiendo— y un semáforo por filósofo; un filósofo solo pasa a comer si ninguno de sus vecinos está comiendo, y al terminar comprueba si puede despertar a sus vecinos hambrientos. El problema ilustra de forma elegante la tensión entre evitar el interbloqueo y aprovechar al máximo los recursos compartidos.

La tabla siguiente resume los tres problemas clásicos, la situación que modelan y el mecanismo de sincronización con que suelen resolverse.

Problema	Situación que modela	Mecanismo habitual
Productor-consumidor	Coordinar la producción y el consumo a través de un búfer acotado	Tres semáforos (vacías, llenas, mutex)
Lectores y escritores	Acceso concurrente a una base de datos compartida	Contador de lectores y semáforo de exclusión
Filósofos comelones	Asignación de recursos compartidos que pueden causar interbloqueo	Semáforo por proceso y arreglo de estados

concurrencia - Tabla 5. *Problemas clásicos de comunicación entre procesos.*

8. Interbloqueos

Los mecanismos de sincronización resuelven las condiciones de carrera, pero introducen un riesgo nuevo: el **interbloqueo** (deadlock). Se produce cuando un conjunto de procesos queda bloqueado de forma permanente, cada uno esperando un recurso que posee otro proceso del mismo conjunto. Ninguno puede avanzar, y sin intervención externa la situación se prolonga indefinidamente (Tanenbaum, 2009).

8.1 Las cuatro condiciones de Coffman

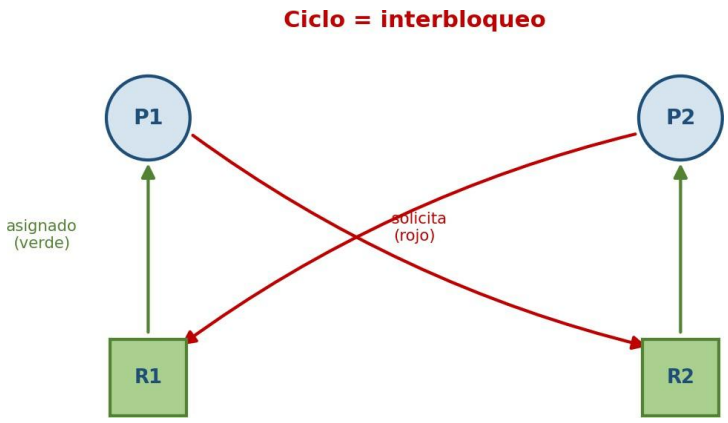
Coffman y sus colaboradores demostraron que un interbloqueo de recursos solo puede ocurrir si se cumplen simultáneamente cuatro condiciones. Si se logra impedir cualquiera de ellas, el interbloqueo se vuelve imposible; esta observación es la base de las estrategias de prevención.

Condición	Descripción
Exclusión mutua	Cada recurso está asignado a un único proceso o se encuentra disponible.
Retención y espera	Un proceso que retiene recursos puede solicitar y esperar por otros nuevos.
Sin expropiación	Los recursos concedidos no pueden retirarse por la fuerza; solo los libera su poseedor.
Espera circular	Existe una cadena circular de procesos, cada uno esperando un recurso que retiene el siguiente.

concurrencia - Tabla 6. *Las cuatro condiciones de Coffman para el interbloqueo.*

8.2 Modelado con grafos de asignación de recursos

El interbloqueo puede representarse mediante un **grafo de asignación de recursos**, en el que los procesos se dibujan como círculos y los recursos como cuadrados. Una arista que va de un recurso a un proceso indica que el recurso está asignado a ese proceso; una arista que va de un proceso a un recurso indica que el proceso lo solicita. Cuando el grafo contiene un ciclo, y hay un solo ejemplar de cada recurso implicado, existe un interbloqueo (Silberschatz et al., 2018).



concurrencia-imagen 6. *Grafo de asignación de recursos con un ciclo de interbloqueo.*

En el grafo anterior, el proceso P1 retiene el recurso R1 y solicita R2, mientras que P2 retiene R2 y solicita R1. La cadena forma un ciclo P1 → R2 → P2 → R1
→ P1 que satisface la condición de espera circular: ambos procesos quedan bloqueados a la espera de un recurso que el otro nunca liberará. El análisis de estos grafos permite tanto detectar interbloqueos ya ocurridos como razonar sobre estrategias para evitarlos.

8.3 Estrategias de manejo

Existen cuatro grandes enfoques para lidiar con el interbloqueo, que difieren en su costo y en el grado de garantía que ofrecen (Tanenbaum, 2009):

1. **Ignorar el problema.** Conocida como la política del avestruz, consiste en asumir que los interbloqueos son tan poco frecuentes que no vale la pena el costo de prevenirlos. Muchos sistemas de propósito general adoptan esta postura pragmática.
2. **Detección y recuperación.** Se permite que ocurran interbloqueos, se detectan analizando el grafo de asignación de recursos y se recuperan, por ejemplo, abortando procesos o expropiando recursos.
3. **Evitación dinámica.** Mediante información sobre las necesidades futuras de recursos, el sistema decide en cada solicitud si concederla mantiene al sistema en un estado seguro. El algoritmo del banquero, de Dijkstra, es el ejemplo canónico.
4. **Prevención estructural.** Se diseña el sistema de modo que alguna de las cuatro condiciones de Coffman no pueda cumplirse nunca, eliminando la posibilidad misma del interbloqueo.

La técnica de prevención más habitual consiste en atacar la espera circular imponiendo un **orden total** sobre los recursos y exigiendo que cada proceso los solicite en orden ascendente. Si todos respetan la misma jerarquía, no puede formarse un ciclo. Esta disciplina resuelve, por ejemplo, el problema de los filósofos comelones: basta con que uno de ellos tome primero el tenedor de mayor número para romper la simetría que conduce al bloqueo.

8.4 Inanición y bloqueo activo

El interbloqueo no es el único peligro de vivacidad. La **inanición** (starvation) ocurre cuando un proceso, aun sin estar bloqueado formalmente, nunca llega a obtener los recursos que necesita porque otros lo adelantan de manera indefinida; es lo que puede sucederle a un escritor en el problema de los lectores y escritores. El **bloqueo activo** (livelock), por su parte, describe una situación en la que los procesos no están bloqueados —siguen ejecutándose y consumiendo CPU— pero tampoco progresan, pues reaccionan continuamente unos a otros sin avanzar, como dos personas que se apartan simultáneamente hacia el mismo lado al cruzarse en un pasillo (Silberschatz et al., 2018). Un buen diseño concurrente debe atender no solo la exclusión mutua, sino también estas garantías de progreso.

9. Concurrencia en los sistemas operativos modernos

Los conceptos estudiados no son meras abstracciones académicas: constituyen la base sobre la que se construyen los sistemas operativos que utilizamos a diario. A continuación se comenta brevemente cómo materializan la concurrencia dos de los sistemas más difundidos, Linux y Windows, así como las tendencias que marcan su evolución.

9.1 Concurrencia en Linux

Linux no distingue tajantemente entre procesos e hilos: ambos se representan mediante una misma estructura interna, la `task_struct`, y se crean con la llamada al sistema `clone`, que recibe un conjunto de banderas para indicar qué recursos — espacio de memoria, archivos abiertos, manejadores de señales— se comparten entre la tarea creadora y la creada. Si no se comparte nada, el resultado se asemeja a un proceso tradicional; si se comparte todo, se obtiene un hilo (Silberschatz et al., 2018).

Internamente, el núcleo de Linux es completamente expropiativo y ofrece un rico repertorio de primitivas de sincronización: candados de giro para las secciones críticas muy breves en contextos donde no se puede dormir, semáforos y mutexes para las que sí lo permiten, y mecanismos avanzados como los *read-*

copy-update (RCU), que optimizan el caso frecuente de muchas lecturas y pocas escrituras. La elección del mecanismo adecuado según la duración de la sección crítica y el contexto de ejecución es una de las claves del rendimiento del núcleo.

9.2 Concurrencia en Windows

La familia Windows implementa el modelo de correspondencia uno a uno, en el que cada hilo de usuario se asocia a un hilo del núcleo. Los hilos se crean con la función `CreateThread` de la API Win32, y su estado se describe mediante estructuras internas del núcleo. Para la sincronización, Windows ofrece objetos de exclusión mutua (mutex), semáforos, eventos y las llamadas *secciones críticas*, que son un mecanismo ligero optimizado para la sincronización entre hilos de un mismo proceso (Silberschatz et al., 2018).

Un rasgo distintivo de Windows es su facilidad de **espera sobre múltiples objetos**: funciones como `WaitForMultipleObjects` permiten que un hilo se bloquee hasta que se cumpla una condición sobre un conjunto de objetos de sincronización, lo que simplifica la coordinación de actividades complejas. Windows adopta, además, una arquitectura híbrida en la que ciertos subsistemas se ejecutan como procesos de usuario que se comunican mediante el paso de mensajes.

9.3 Tendencias y desafíos

La generalización de los procesadores multinúcleo ha situado a la concurrencia en el centro del diseño de software. Frente a la dificultad de programar con cerrojos, han cobrado fuerza enfoques que abstraen la gestión de hilos: los **conjuntos de hilos** (thread pools), que reutilizan un número acotado de hilos para atender muchas tareas; los marcos de **bifurcación-unión** (fork-join), idóneos para algoritmos de divide y vencerás; y sistemas como Grand Central Dispatch de Apple o las bibliotecas Intel TBB, que permiten al programador identificar tareas paralelizables y delegan en la biblioteca la creación y planificación de los hilos (Silberschatz et al., 2018).

Otra línea de investigación busca reducir la dependencia de los cerrojos mediante estructuras de datos **sin bloqueo**, construidas sobre instrucciones atómicas como compare-and-swap, y mediante la **memoria transaccional**, que permite agrupar operaciones sobre memoria compartida en transacciones que se ejecutan de forma atómica y se reintentan si entran en conflicto. Estas técnicas prometen simplificar la programación concurrente y mejorar la escalabilidad, aunque su adopción plena aún plantea retos de rendimiento y de soporte de hardware. En cualquier caso, el dominio de los fundamentos expuestos en esta investigación seguirá siendo imprescindible para comprender y aprovechar las arquitecturas del futuro.

10. Conclusión

La concurrencia constituye uno de los pilares conceptuales de los sistemas operativos modernos y, a la vez, una de sus mayores fuentes de complejidad. A lo largo de esta investigación se ha recorrido un camino que parte de las nociones fundamentales de proceso e hilo, atraviesa el problema central de la sección crítica y culmina en los mecanismos y estrategias que permiten construir programas concurrentes correctos.

Del recorrido se desprenden varias ideas centrales. En primer lugar, la concurrencia y el paralelismo, aunque relacionados, no son lo mismo: la primera estructura la ejecución de múltiples actividades, mientras que el segundo aprovecha varios núcleos para acelerarlas. En segundo lugar, el acceso no coordinado a datos compartidos da lugar a condiciones de carrera, cuya prevención exige garantizar la exclusión mutua sobre las regiones críticas, respetando además los requisitos de progreso y de espera acotada.

En tercer lugar, existe una jerarquía de soluciones que evoluciona desde las técnicas de software, como la solución de Peterson, pasando por el apoyo del hardware mediante instrucciones atómicas, hasta los mecanismos de alto nivel — semáforos, mutexes, variables de condición, monitores, mensajes y barreras—que evitan la espera ocupada y facilitan la programación. Cada mecanismo ofrece un compromiso distinto entre eficiencia, expresividad y seguridad, por lo que la elección adecuada depende del contexto.

En cuarto lugar, los problemas clásicos —productor-consumidor, lectores y escritores, y filósofos comelones— condensan las dificultades esenciales de la coordinación y sirven de piedra de toque para cualquier mecanismo de sincronización. Finalmente, el interbloqueo, la inanición y el bloqueo activo recuerdan que la corrección de un programa concurrente no se agota en la exclusión mutua: también debe garantizarse el progreso del sistema en su conjunto.

En un panorama tecnológico dominado por los procesadores multinúcleo, el dominio de estos principios ha dejado de ser un conocimiento especializado para convertirse en una competencia indispensable de todo profesional de la informática. Comprender cómo coordinar actividades que comparten recursos — evitando tanto las carreras como los bloqueos— es la condición necesaria para escribir software capaz de aprovechar el hardware contemporáneo de manera correcta, eficiente y escalable.

Referencias bibliográficas

Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (2014). *Operating systems: Three easy pieces* (Versión 0.80). Arpaci-Dusseau Books.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). John Wiley & Sons.

Tanenbaum, A. S. (2009). *Sistemas operativos modernos* (3.ª ed.). Pearson Educación.